



Instituto Superior Politécnico de Tecnologias e Ciências
(ISPTEC)

Departamento de Engenharia e Tecnologias

Base de Dados II

Licenciatura em Engenharia Informática

2025/2026

Lab #02 — Exploração de Banco de Dados Distribuídos

Isabel Marques — 20231238
Jussana Paim — 20230132
Norberto Cassoma — 20230873
Oldmar Filindo — 20231359

LUANDA, 2026

PARTE 1 — EXPLORAÇÃO GUIADA

Fase 0 — Criação do Dataset MercadoKwanza

REGISTO DE RESULTADOS — FASE-0

N.º de linhas em VENDA após importação	30000
N.º de linhas em ITEM_VENDA	90000
N.º de clientes importados	5000
N.º de produtos importados	200
Erros encontrados no script da IA (s/n — quantos?)	O script gerado pela IA inseriu os valores de população das províncias em formato decimal em vez de inteiro. Por exemplo, estava a inserir 9.292000 em vez de 9292000. Como o atributo populacao foi definido como INT, o valor decimal era inválido ou ficava truncado

REFLEXÃO ESCRITA — resposta individual no relatório

1. O script gerado pela IA funcionou directamente ou precisou de correcções? Que tipo de erros surgiram e o que aprenderam ao corrigi-los?

R: O script não foi executado directamente. O grupo fez primeiro uma análise do código gerado e identificou dois problemas antes de executar. O primeiro problema foi de normalização. A IA gerou o campo categoria como um simples texto VARCHAR dentro da tabela PRODUTO, o que viola a 3ª Forma Normal. O grupo decidiu criar uma tabela separada CATEGORIA com chave estrangeira, garantindo integridade referencial e eliminando redundância de dados. O segundo problema foi nos valores de população das províncias. A IA inseriu valores decimais como 9.292000 em vez de inteiros como 9292000. Como o campo foi definido como INT, os valores ficavam incorrectos. O grupo corrigiu para os valores inteiros reais.

2. Porque é que o grupo decidiu usar stored procedures com loops em vez de INSERT linha a linha? Qual a vantagem em termos de tempo e manutenção?

R:Decidimos usar por causa da automatizacao, pois ao inves de inserirmos dados por dados que nos levarias muito tempo usar as stored por uma questao mais de perfomacae com isso se tranalha com mais eficiencia e pelo fato de usar o stored que basicamente sao bloco de codgo guardados na base de dados , por permite reutilizar os blocos , vamtagem sobre a mantuencao e que permitira facil alteraca num bloco de codigo de forma facil e fato de elas serem ou poderem ser parametrizada da essa opcao de melhorar e indo mudando os resultado .

Fase 1 — O Docker e a Ideia de Nó Distribuído

REGISTO DE RESULTADOS — FASE-1A

Versão do Docker instalada	PS C:\Users\USER> docker --version Docker version 29.5.3, build d1c06ef
Versão do Docker Compose	PS C:\Users\USER> docker compose version Docker Compose version v5.1.4
Sistema Operativo usado pelo grupo	Windows 11, versao 25H2

REGISTO DE RESULTADOS — FASE-1B

N.º de contentores com STATUS Up	4
Tempo de arranque do cluster (aprox.)	07:46:10 tempo de arranque e 07:46:37 tempo que terminou Total da 27 segundos
Resultado de COUNT(*) FROM VENDA (Luanda)	30000
Resultado de COUNT(*) FROM CLIENTE (Luanda)	5000
Os dados do SQL foram carregados automaticamente? (S/N)	Não de forma imediata. Na primeira tentativa o contentor carregou apenas uma parte dos dados e o COUNT da tabela VENDA retornou um valor incorrecto. Foi necessário parar o cluster com docker compose down e levantar novamente com docker compose up. Depois esperámos aproximadamente 10 minutos para o carregamento completo. Acompanhámos o progresso com o comando docker logs no-luanda e só executámos as queries de verificação quando apareceu a mensagem ready for connections nos logs. Após essa confirmação os dados estavam correctos com 30 000 vendas e 5 000 clientes.

REFLEXÃO ESCRITA — resposta individual no relatório

1. O que significa o mapeamento de portas '3307:3306'? Porque é que os nós Benguela e Huambo não podem também usar a porta 3306 no anfitrião?

R: O mapeamento 3307:3306 significa que a porta 3307 do nosso PC é redireccionada para a porta 3306 dentro do contentor. Os nós Benguela e Huambo não podem usar a porta 3306 no anfitrião porque os três contentores estão no mesmo PC e uma porta só pode ser usada por um serviço de cada vez, haveria conflito. Por isso cada nó tem uma porta diferente no anfitrião mas internamente dentro da rede Docker todos comunicam entre si pela porta 3306 pois esse é o canal interno de comunicação entre eles sem conflito porque estão em contentores isolados.

2. Qual a vantagem de usar o volume './dados:/docker-entryptpoint-initdb.d'? O que aconteceria se não usassem esse volume e quisessem carregar os dados manualmente?

R: A vantagem do volume é a optimização no carregamento dos dados. O MySQL detecta automaticamente os ficheiros SQL na pasta docker-entryptpoint-initdb.d e executa-os quando o contentor arranca pela primeira vez sem necessidade de intervenção manual. Sem esse volume teríamos de entrar contentor por contentor manualmente com docker exec e carregar os dados de forma sequencial o que seria mais demorado e sujeito a erros. Com o volume o processo é automático e cada contentor trata do seu próprio carregamento de forma independente no arranque.

3. Se desligarem o computador e voltarem a ligar, os dados permanecem? Testem e documentem o que observam.

R: Sim, os dados permanecem após desligar e ligar o PC. Isto acontece porque os dados ficam

guardados no volume do Docker no disco do computador. No entanto os contentores não arrancam automaticamente sendo necessário abrir o Docker Desktop e executar docker compose up para os ligar novamente. Após isso verificámos que os dados continuavam intactos com 30 000 vendas e 5 000 clientes confirmando que o volume preserva os dados entre sessões. Os dados só seriam apagados se executássemos docker compose down com a opção v que remove os volumes.

Fase 2 — Arquitectura Distribuída e Fragmentação

REGISTO DE RESULTADOS — FASE-2

Total vendas — fragmento Luanda	10000
Total vendas — fragmento Benguela	10000
Total vendas — fragmento Huambo	10000
Soma dos 3 fragmentos = total? (S/N)	30000
Query de fragmentação vertical escrita pelo grupo? (S/N)	S

REFLEXÃO ESCRITA — resposta individual no relatório

1. Segundo Özsü & Valduriez (2020), um esquema de fragmentação válido deve respeitar três propriedades: completude, disjunção e reconstituição. As views que criaram respeitam estas três propriedades? Justifiquem para cada uma.

R: A soma dos três fragmentos deu exactamente 30 000, que é o total da tabela venda. Isso significa que nenhuma venda ficou de fora [completude]. Como cada loja pertence a uma única província, uma venda não pode aparecer em dois fragmentos ao mesmo tempo [disjunção]. E se fizemos um UNION ALL das três views, recuperamos a tabela original [reconstituição].

2. Quais são as limitações de simular fragmentação com VIEWS em vez de distribuir fisicamente os dados por diferentes servidores? O que é que as VIEWS não conseguem simular?

R: As views foram todas criadas no nó de Luanda, não em servidores diferentes. Na prática isso significa que não há fragmentação física real — se o servidor de Luanda cair, todos os fragmentos ficam inacessíveis ao mesmo tempo. Com replicação, os outros nós teriam os dados, mas as views em si continuavam a ser um conceito lógico, não físico.

3. Num cenário real, que factores decidem se uma tabela deve ser fragmentada horizontalmente, verticalmente, ou não fragmentada de todo?

R: Faz sentido fragmentar por província porque cada loja acede principalmente aos seus próprios dados. Tabelas pequenas e globais como província ou categoria devem existir em todos os nós porque são consultadas por toda a gente.

Fase 3 — Replicação MySQL: Master e Slaves

REGISTO DE RESULTADOS — FASE-3

Slave_IO_Running — Benguela	Yes
Slave_SQL_Running — Benguela	Yes
Seconds_Behind_Master — Benguela	0
Slave_IO_Running — Huambo	Yes
Seconds_Behind_Master — Huambo	0
Os 3 produtos inseridos aparecem no Slave Benguela? (S/N)	S
Os 3 produtos inseridos aparecem no Slave Huambo? (S/N)	S

REGISTO DE RESULTADOS — FASE-3B

O que aconteceu às leituras no Slave quando o Master estava pausado?	Funcionaram normalmente
O que aconteceu quando tentaram escrever com o Master pausado?	Deu o erro: "Container no-luanda is paused"
Após restaurar o Master, o Slave sincronizou automaticamente? (S/N)	S

REFLEXÃO ESCRITA — resposta individual no relatório

1. A replicação Master-Slave é síncrona ou assíncrona? O que significa isso para a consistência dos dados? Se um cliente comprar um produto em Luanda (Master) e imediatamente consultar o stock em Benguela (Slave), pode ver um valor desactualizado?

R: A replicação é assíncrona. Isso significa que pode existir um momento curto em que o Slave ainda não tem os dados mais recentes. Se um cliente comprar em Luanda e consultar o stock em Benguela imediatamente a seguir, pode ver um valor desactualizado.

2. Qual o papel do binary log (binlog) na replicação? O que aconteceria se o binary log fosse apagado acidentalmente no Master enquanto os Slaves ainda não tinham lido todas as entradas?

R: O binary log é onde o Master regista tudo o que acontece. Os Slaves lêem ficheiro e repetem as mesmas operações. Se o for o log gado antes de um Slave ler, esse Slave perde a sincronização e teria de ser refeito do zero.

3. Numa arquitectura real com 10 Slaves, o que acontece se o Master cair permanentemente? Que estratégia poderia o grupo adoptar para tornar o sistema mais resiliente?

R: Numa situação real, poderia ser necessário promover um dos Slaves a Master, escolhendo aquele com Seconds_Behind_Master igual a zero.

Referências Bibliográficas

- ÖZSU, M. T.; VALDURIEZ, P. (2020). Principles of Distributed Database Systems. 4.ª ed. Springer. — Referência principal sobre fragmentação, replicação e SGBDD.
- BREWER, E. A. (2000). Towards Robust Distributed Systems. Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC). ACM. — Formulação original do Teorema CAP.
- GILBERT, S.; LYNCH, N. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, 33(2). — Prova formal do Teorema CAP.
- GRAY, J.; LAMPORT, L. (2006). Consensus on Transaction Commit. ACM Transactions on Database Systems, 31(1), pp. 133–160. — Two-Phase Commit e Paxos.
- KLEPPMANN, M. (2017). Designing Data-Intensive Applications. O'Reilly Media. — Capítulos 5, 6 e 9: replicação, particionamento e transações distribuídas.
- CHODOROW, K. (2019). MongoDB: The Definitive Guide. 3.ª ed. O'Reilly Media. — Modelo de documentos, sharding e agregações.
- COULOURIS, G. et al. (2012). Distributed Systems: Concepts and Design. 5.ª ed. Addison-Wesley. — Fundamentos de sistemas distribuídos e tolerância a falhas.
- MySQL 8.0 Reference Manual. Oracle Corporation, 2024. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>
- MongoDB Manual 7.0. MongoDB, Inc., 2024. Disponível em: <https://www.mongodb.com/docs/manual/>

— Bom trabalho, e boas descobertas! —

"Um sistema distribuído é aquele em que a falha de um computador que você nem sabia que existia pode tornar o seu computador inutilizável."

— Leslie Lamport